Reviewer Pack — Minimal Rank of Tropicalized Quandle Operations vs Communica…

Entry dbb9d64daa0c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 19:54:11 UTC

Minimal Rank of Tropicalized Quandle Operations vs Communication Complexity for Disjointness

Entry ID: `dbb9d64daa0c` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 19:54:11 UTC

1. Conjecture

Field A (mathematical branch): Quandle Theory **Field B** (complexity object): Communication Complexity

Statement:

> ['For every n-ary quandle Q with minimal rank r, there exists a disjointness instance such that the randomized communication complexity of the partial function defined by Q is Ω(n^r).', 'Equivalently, for any n and δ > 0, if the communication complexity of the disjointness problem is less than (n^r)/δ, then the minimal rank of the quandle Q must be at least r.', 'This lower bound holds with high probability for randomly generated quandles.']

Rationale (proposer's reasoning):

> ['Quandle theory provides a rich algebraic structure that may encode complex combinatorial properties. The tropicalization process allows us to map these structures into a semiring, potentially preserving complexity-theoretic properties.', 'Disjointness is a fundamental problem in communication complexity with known lower bounds. If quandle operations can provide strong lower bounds for disjointness, it would suggest a deep connection between algebraic structures and communication complexity.']

Taxonomy category: `COMM_DISJ` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `d4895dc995699751`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Evaluate the randomized communication complexity for disjointness on randomly generated n-ary quandles. If the average communication complexity across all seeds is at least (n^r)/δ, then support the conjecture; if any seed produces a complexity greater than (n^r)/(2δ), the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_quandle(n, r):
        quandle = []
        for i in range(r):
            row = [0] * n
            for j in range(i + 1, n):
                row[j] = random.randint(0, n - 1)
            quandle.append(row)
        return quandle

    def communication_complexity(quandle, n):
        complexity = 0
        for i in range(n):
            for j in range(i + 1, n):
                if quandle[i][j] == quandle[j][i]:
                    complexity += 1
        return complexity

    n = random.randint(5, 40)
    r = random.randint(1, min(3, n - 1))
    quandle = generate_quandle(n, r)
    cc = communication_complexity(quandle, n)

    metric_value = cc / (n ** r)
    return {
        "metric_name": "Communication Complexity",
        "metric_value": metric_value,
        "instances_tested": 1,
```

```
        "conjecture_holds": True if metric_value >= 1 else False,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fractior
    elif any(not r["conjecture_holds"] and r["metric_value"] > 1 / (2 * min(3, n - 1)) for r in re
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='communication_complexity_too_low' first_failing_
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0f41438a.py", line 57, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0f41438a.py", line 41, in run_trial
    cc = communication_complexity(quandle, n)
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0f41438a.py", line 34, in communication
    if quandle[i][j] == quandle[j][i]:
                        ~~~~~~~~^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing any data, which means we cannot evaluate the conjecture's support or falsification based on the pre-registered criteria. | next: Re-run the test with proper error handling to ensure it completes without crashing and produces results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15543
2	propose	ollama_remote	glm4:latest	0	0	13701
3	propose	ollama_remote	glm4:latest	0	0	13133
4	preregistration	ollama_remote	glm4:latest	0	0	9041
5	novelty	ollama_remote	glm4:latest	0	0	11734
6	novelty	ollama_remote	glm4:latest	0	0	8538
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16367
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10185
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9841
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7123
11	judge	ollama_remote	glm4:latest	0	0	11691

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 126898 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/dbb9d64daa0c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/dbb9d64daa0c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/dbb9d64daa0c.tar.gz` (if generated)